

Dossier Projet : Bachelor Studi Administrateur système Devops

Présenté par : Enzo Loppin-Antimi

The logo for 'Studi' is displayed in a large, bold, sans-serif font. The word 'studi' is in dark grey, and the 'di' is in a bright orange color. A small orange triangle is positioned above the 'i'.

3. Sommaire

Dossier Projet : Bachelor Studi Administrateur système Devops.....	1
3. Sommaire.....	2
4. Résumé.....	3
5. Présentation de la formation d'administrateur système devops :.....	5
6. Contexte & mission.....	6
7. Cahier des charges.....	9
8. Cadre de référence.....	11
9. Architecture cible.....	11
10.1 Module – Comptes & Security Baseline.....	18
10 .2 Module – Core Network.....	22
10 .3 Module - Shared Core (S3 & KMS).....	28
10 .4 Module Ingress (ACM, ALB, WAF, DNS).....	30
10.5 Module – EKS.....	32
10.6 Module – RDS (PostgreSQL).....	35
10.7 Module – ECR.....	37
10.8 Module CI/CD Infra.....	39
10.9 Module - Secrets Management.....	42
11.Configuration des Environnements.....	45
12 Déploiement continue (CI/CD + conteneurisation).....	52
12. Observabilité & exploitabilité.....	57
13. Sauvegardes & PRA.....	61
14 . Synthèse de la réalisation & Axes d'amélioration futurs.....	64

4. Résumé

Résumé en français

Le projet RHZORION vise la conception et le déploiement d'une plateforme SaaS de gestion des ressources humaines, construite dès l'origine selon une approche **cloud-native** sur AWS. L'architecture retenue repose sur une organisation multi-comptes et multi-AZ, permettant d'assurer la disponibilité, la sécurité et la conformité réglementaire (RGPD, référentiels ANSSI). La démarche s'inscrit dans un cadre **DevSecOps**, intégrant l'automatisation de l'infrastructure (Terraform), une chaîne CI/CD sécurisée (GitLab, OIDC, analyses automatisées, signatures), ainsi qu'une supervision unifiée (Prometheus, Grafana, Fluent Bit, OpenTelemetry). Des stratégies de sauvegarde et de PRA complètent ce dispositif. L'ensemble permet d'offrir une solution conforme, traçable et résiliente, conciliant agilité et exigences de confiance.

Abstract in English

The RHZORION project focuses on the design and deployment of a Human Resources SaaS platform, developed from the beginning with a **cloud-native** approach on AWS. The target architecture is based on a multi-account and multi-AZ organization, ensuring availability, security, and compliance with regulations (GDPR, ANSSI guidelines). The methodology adopts a **DevSecOps** framework, integrate infrastructure automation (Terraform), a secure CI/CD pipeline (GitLab, OIDC, automated scans, signatures), and unified observability (Prometheus, Grafana, Fluent Bit, OpenTelemetry). Backup strategies and disaster recovery procedures complement this setup. The overall objective is to deliver a compliant, auditable, and resilient solution, combining organizational agility with enterprise-level reliability.

5. Présentation de la formation d'administrateur système devops :

La formation proposée par l'école Studi en Administration Système DevOps est structurée autour de trois Certificats de Compétences Professionnelles (CCP), visant à couvrir l'intégralité du cycle de vie d'une infrastructure informatique moderne :

- CCP1 : Automatiser le déploiement d'une infrastructure dans le cloud – maîtrise des outils de provisionnement, gestion d'infrastructure par code et automatisation complète.
- CCP2 : Déployer en continu une application – intégration et livraison continues, déploiement automatisé des applications en environnement cloud.
- CCP3 : Superviser les services déployés – mise en place et gestion proactive du monitoring et de l'observabilité des systèmes.

<https://www.francecompetences.fr/recherche/rncp/36061/>



6. Contexte & mission

6.1 Présentation de l'entreprise RHZORION

RHorizon est une filiale du groupe Horizon Technologies, créée en 2025 pour répondre aux besoins croissants des PME en matière de digitalisation des ressources humaines.

Le choix stratégique a été de concevoir la plateforme **dès le départ en mode cloud-native**, en intégrant sécurité, scalabilité et conformité (RGPD, ANSSI DevSecOps v1.0).

RHZORION est une application SaaS de gestion des ressources humaines 100% web. Elle permet aux PME/ETI de :

- Gérer les congés et absences : demandes en ligne, calendriers partagés, workflows d'approbation
- Suivre le temps de travail : saisie d'heures, validation de feuilles de présence , reporting
- Centraliser les documents RH : stockage sécurisé des contrats, bulletins de paie, certificats en tout genre
- Piloter la stratégie RH via des dashboards : indicateurs clés RH (turnover, coûts, présence)
- Préparer les intégrations : ouverture future vers APIs de paie, formation, données externes

L'application repose sur une architecture applicative simple :

- Frontend : interface web responsive (Vue.js + Nginx), accessible depuis n'importe quel navigateur
- Backend : moteur métier exposé via API REST (Flask + Gunicorn), qui traite les règles RH et les demandes
- Base de données : persistance des données (PostgreSQL), stockage des congés, heures, infos utilisateurs
- Stockage objet : documents scannés ou uploadés (S3), accessible et immuable pour audit

6.2 Rôle de l'administrateur DevOps

L'administrateur DevOps intervient en amont du lancement du produit pour :

- concevoir et déployer l'infrastructure cloud
- mettre en place une chaîne **CI/CD sécurisée**
- intégrer la **sécurité dès la conception**
- assurer la **supervision et observabilité** continue,
- automatiser et industrialiser l'infrastructure

6.3 Cadre Juridique et Normatif

En tant qu'entreprise européenne recueillant de données personnelles, le projet est soumis à des obligations strictes qui conditionnent l'architecture :

RGPD

- Souveraineté : Données en Union Européenne uniquement
- Chiffrement : Obligatoire au repos et en transit
- Privacy by Design : Sécurité dès la conception
- Droit d'audit : Traçabilité complète des accès
- Droit à l'oubli : Suppression possible des données

ANSSI DevSecOps v1.0

- Séparation stricte environnements (dev ≠ prod)
- MFA obligatoire pour accès administrateur
- Gestion secrets sécurisée, rotation automatique
- Scans sécurité automatisés (code, infra, conteneurs)
- Logs immuables et traçables

AWS Well-Architected Framework

5 piliers : Excellence Opérationnelle, Sécurité, Fiabilité, Efficacité Performance, Optimisation Coûts.

7. Cahier des Charges

7.1 Périmètre Technique

L'infrastructure doit supporter :

Couche applicative : Frontend, backend, orchestration conteneurs, scalabilité 1-100+ instances.

Données structurées : Base relationnelle pour métier (congrés, heures, utilisateurs), transactions garanties, sauvegardes régulières, HA en prod, restauration point-in-time.

Documents : Stockage en masse chiffré, versioning complet, archivage long-terme économique, accès granulaire.

7.2 Exigences Sécurité & Conformité

Souveraineté : Zéro donnée hors Europe, politique refusant déploiements non-UE.

Confidentialité : Chiffrement obligatoire au repos et en transit, gestion centralisée clés, rotation auto.

Identité & Accès : Authentification centralisée + MFA, zéro secrets persistants, permissions minimales par service, permet un audit de toute action sensible.

Traçabilité : Chaque action loguée, logs immuables (non modifiables 2+ ans), logs centralisés non-supprimables.

Isolation Environnements : Infra séparées développement/production, pipelines distincts, configurations distinctes, secrets distincts.

7.3 Exigences Disponibilité & Opérations

Haute Disponibilité (HA) : Multi-AZ, zéro point de défaillance unique, absorber panne AZ complète. Objectif : 99.9% uptime.

Continuité (PRA) : RPO < 15 min, RTO < 1h, sauvegardes fréquentes (5 min), historique 35+ jours, copies secours, infrastructure redéployable facilement , tests restauration 6 mois.

6.3 Rôle de l'administrateur DevOps

L'administrateur DevOps intervient en amont du lancement du produit pour :

- concevoir et déployer l'infrastructure cloud
- mettre en place une chaîne **CI/CD sécurisée**
- intégrer la **sécurité dès la conception**
- assurer la **supervision et observabilité** continue,
- automatiser et industrialiser l'infrastructure

9. Architecture Cible

9.1 Vue d'ensemble & Philosophie

L'architecture de la plateforme RHZORION repose sur un modèle **Cloud-Native** hébergé sur AWS. Elle a été conçue pour répondre aux exigences d'une application SaaS RH : scalabilité, haute disponibilité et conformité stricte (RGPD, ANSSI). L'infrastructure s'inscrit dans une démarche **DevSecOps** : tout est déployé via du code (Infrastructure as Code), la sécurité est intégrée dès la conception (*Security by Design*) et les opérations sont automatisées via des pipelines CI/CD.

9.2 Fondations Cloud & Organisation (Landing Zone)

L'organisation AWS est structurée en **multi-comptes** pour garantir une isolation forte des environnements et limiter le rayon d'impact en cas d'incident.

- **Shared-Services (sh)** : Compte "Hub" centralisant les services transverses : registres de conteneurs (ECR), clés de chiffrement (KMS), DNS (Route53) et surtout le stockage des journaux d'audit inaltérables.
- **Non-Prod (np)** : Environnement de développement et de recette. L'architecture y est optimisée pour maîtriser les coûts (ressources limitées).
- **Prod (pr)** : Environnement de production sanctuarisé. Il garantit la haute disponibilité (Multi-AZ) et la performance pour les utilisateurs finaux.

Gouvernance et Identité :

- L'accès est exclusivement fédéré via **IAM Identity Center (SSO)** avec authentification multi-facteurs (MFA) obligatoire.
- Aucun utilisateur IAM permanent n'est créé. Les administrateurs et les pipelines CI/CD utilisent des rôles temporaires.
- Des **SCPs (Service Control Policies)** verrouillent le périmètre : interdiction de sortir de l'UE (RGPD) et interdiction de désactiver les logs d'audit (CloudTrail).

9.3 Core Network : Isolation et Connectivité

Le réseau est segmenté via un **VPC** dédié par environnement, conçu en trois tiers pour une défense en profondeur:

1. **Subnets Publics** : Hébergent uniquement les Load Balancers et les NAT Gateways.
2. **Subnets Applicatifs (Privés)** : Hébergent les nœuds EKS (Frontend/Backend). Ils n'ont aucune exposition directe à Internet.
3. **Subnets Data (Isolés)** : Hébergent les bases de données, accessibles uniquement depuis la couche applicative.

Connectivité Sécurisée : Les communications vers les services managés AWS (S3, ECR, KMS) passent par des **VPC Endpoints** (PrivateLink), garantissant que le trafic reste sur le réseau privé AWS sans passer par Internet.

9.4 Plateforme Applicatif : EKS & Conteneurisation

L'orchestration des conteneurs est confiée à **Amazon EKS** (Elastic Kubernetes Service) pour assurer la scalabilité et la maintenance simplifiée.

- **Sécurité des Workloads** : Les conteneurs (Vue.js et Flask) s'exécutent en utilisateur non-root et utilisent des images figées (pas de tag `:latest`).
- **Gestion des accès (IRSA)** : L'authentification aux services AWS se fait via *IAM Roles for Service Accounts*. Chaque pod possède une identité propre, éliminant le besoin de clés d'accès statiques.
- **Flux internes** : Des *NetworkPolicies* appliquent une politique "deny-by-default", n'autorisant que les flux explicitement nécessaires entre le frontend, le backend et la base de données. **Stockage**

9.5 Stratégie de Données & (Data Layer)

La persistance des données est assurée par deux services distincts, garantissant intégrité et résilience :

- **Bases de données (RDS PostgreSQL)** : Le service est déployé en Multi-AZ en production pour la haute disponibilité. Les données sont chiffrées au repos (KMS) et en transit (SSL). Les sauvegardes sont automatisées avec une rétention de 30 jours et la possibilité de restauration à un instant précis (PITR).
- **Stockage Objet (S3 Centralisé)** : Situé dans le compte `shared-services`, il centralise les logs et les backups. L'option **Object Lock** est activée pour rendre les journaux d'audit immuables, protégeant contre les suppressions malveillantes (ransomwares).

9.6 Ingress

L'exposition de l'application est strictement contrôlée et sécurisée à l'entrée du réseau :

- **Point d'entrée unique** : Un **Application Load Balancer (ALB)** distribue le trafic entrant.
- **Chiffrement TLS** : Tout le trafic est forcé en HTTPS. Les certificats sont gérés et renouvelés automatiquement par AWS ACM.
- **Protection WAF** : Un pare-feu applicatif (**AWS WAFv2**) filtre les requêtes malveillantes (injections SQL, XSS, bots) avant qu'elles n'atteignent le cluster.

9.7 :CI/CD & Supply Chain

Le déploiement est entièrement automatisé via **GitLab CI/CD**, suivant les principes du GitOps et de la sécurité de la supply chain.

- **Authentification OIDC** : GitLab s'authentifie auprès d'AWS via OpenID Connect, supprimant tout stockage de clés secrètes dans les variables du pipeline.
- **Sécurité de la chaîne** : Chaque image Docker est scannée (recherche de vulnérabilités), signée cryptographiquement (Cosign) et accompagnée d'un SBOM (Software Bill of Materials) avant d'être déployée.
- **Déploiement contrôlé** : En production, le déploiement nécessite une approbation manuelle (gate) après validation automatique des tests de sécurité en nonprod

9.8 Observabilité & Continuité (PRA)

Pour garantir la fiabilité du service, une stack complète de supervision et de résilience est en place :

- **Observabilité Unifiée** : **Prometheus** collecte les métriques, **Grafana** permet la visualisation des tableaux de bord, et **Fluent Bit** centralise les logs vers S3 et CloudWatch. **OpenTelemetry** assure le traçage distribué des requêtes backend.
- **Plan de Reprise d'Activité (PRA)** : Des sauvegardes régulières (RDS et S3) sont répliquées vers le compte de services partagés. Des tests de

restauration sont prévus pour valider les objectifs de RPO (15 min) et RTO (2h) en production.

Différenciation des Environnements NonProd et Prod : Stratégie et Justifications

L'infrastructure RHZORION est déclinée en deux environnements distincts : **Non-Prod (NP)** et **Production (PR)**. Cette distinction répond à un arbitrage constant entre optimisation budgétaire et exigences de haute disponibilité.

1. Philosophie Générale

- **Non-Prod (NP)** : Cet environnement est conçu comme une "maquette iso-fonctionnelle" mais allégée. L'objectif prioritaire est de valider les fonctionnalités et l'intégration technique tout en minimisant la facture cloud. Une interruption de service y est tolérable.
- **Production (PR)** : Cet environnement est sanctuarisé. La priorité absolue est la continuité de service ("Zéro Interruption"), la performance pour l'utilisateur final et la conformité stricte aux standards de sécurité (RGPD/ANSSI)²²².

2. Résilience Réseau et Disponibilité

L'architecture réseau varie considérablement pour adapter le niveau de redondance :

- **En Non-Prod** : Le déploiement se limite à **2 Zones de Disponibilité (AZ)** et utilise une **unique NAT Gateway** partagée pour tout le VPC. Ce choix introduit un point de défaillance unique (SPOF) mais permet une économie substantielle sur les coûts horaires de l'infrastructure AWS.
- **En Production** : L'infrastructure s'étend sur **3 Zones de Disponibilité (AZ)** avec une **NAT Gateway dédiée par zone**. Cette redondance totale garantit que la perte complète d'une AZ ou d'une passerelle n'affecte pas la connectivité des autres zones, assurant une résilience maximale

3. Stratégie de Données (RDS & S3)

La gestion des données diffère pour répondre aux objectifs de RPO (Recovery Point Objective) :

- **En Non-Prod** : La base de données RDS est déployée en instance unique (**Single-AZ**) avec une rétention des sauvegardes courte de **7 jours**. C'est une configuration économique suffisante pour le développement.
- **En Production** : RDS est configuré en **Multi-AZ** (réplication synchrone) pour assurer une bascule automatique en cas de panne. La rétention est étendue à **30 jours** avec l'activation du **PITR** (Point-In-Time Recovery) pour permettre une restauration à la seconde près en cas de corruption logique. De plus, les logs d'audit sont verrouillés via **S3 Object Lock** pour garantir leur immutabilité légale.

4. Puissance de Calcul (EKS)

Le dimensionnement du cluster Kubernetes s'adapte à la réalité du trafic :

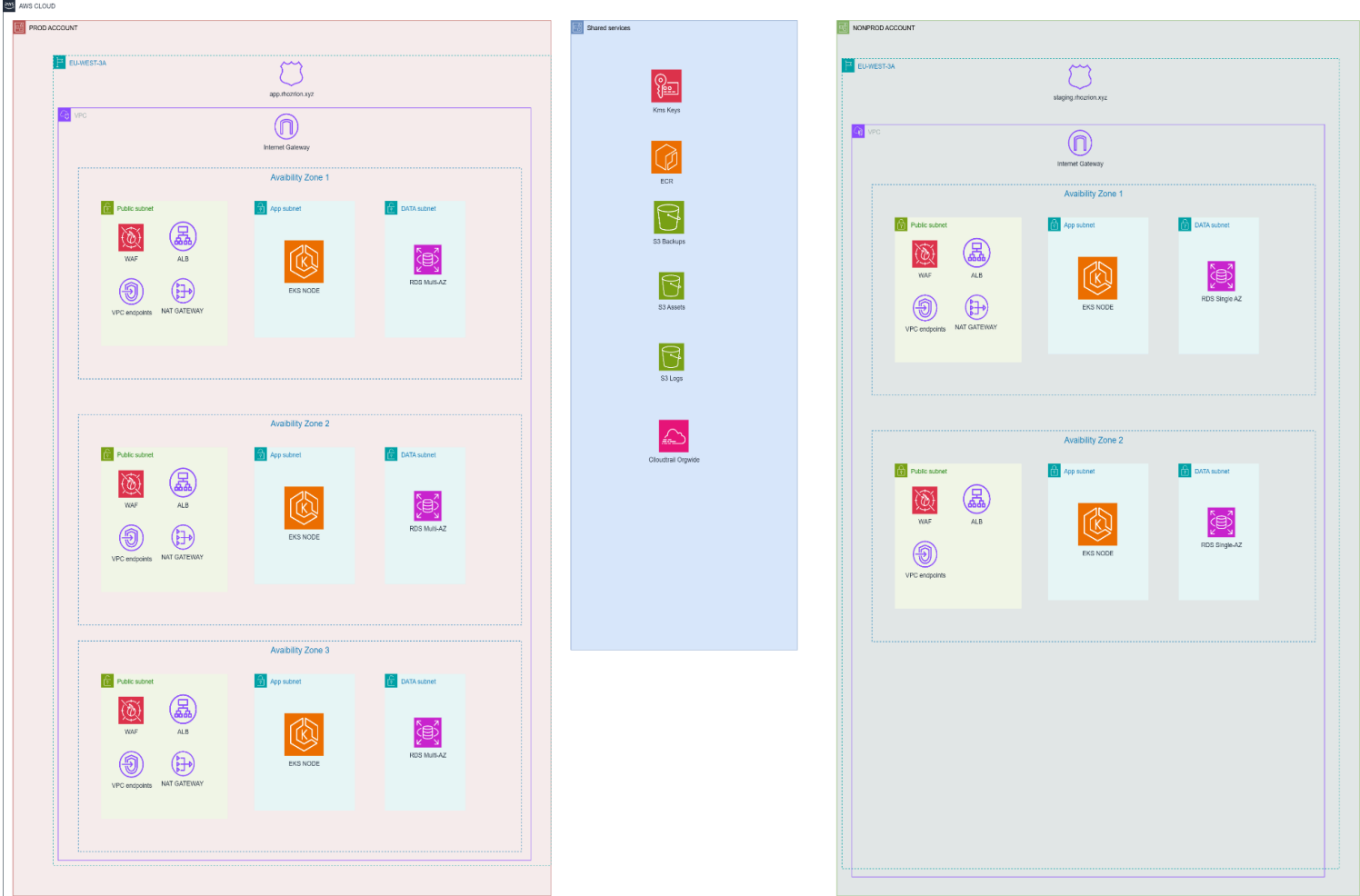
- **En Non-Prod** : Les nœuds utilisent des instances de type **"Burstable" (t3.medium)**. Elles sont moins coûteuses mais offrent des performances CPU variables, adaptées à un trafic intermittent de test.
- **En Production** : Les workloads tournent sur des instances **Standard (m5.large)** garantissant des ressources dédiées et stables. L'auto Scaling est configuré pour absorber les pics de charge réels sans dégrader l'expérience utilisateur.

5. Sécurité et Déploiement

Les contraintes opérationnelles sont ajustées selon la criticité :

- **En Non-Prod** : Le déploiement est **continu et automatique** après validation des tests pour favoriser la vélocité des développeurs. Le WAF (pare-feu) est configuré avec des règles souples pour ne pas bloquer les tests.
- **En Production** : Tout déploiement est soumis à un **"Gate" manuel** nécessitant une double approbation pour éviter les régressions. Le WAF applique un filtrage strict et un *rate-limiting* agressif pour contrer les attaques.

Enzo Loppin-Antimi - Studi Promotion KAO



10. Spécifications des Modules d'Infrastructure

Afin de concrétiser l'architecture cible définie précédemment, le déploiement de l'infrastructure RHZORION repose sur une approche modulaire et standardisée via Terraform.

L'objectif de cette section est de détailler les briques techniques (modules) qui composent la plateforme. Conformément aux principes DevSecOps et DRY (*Don't Repeat Yourself*), chaque module a été conçu de manière agnostique. Cela signifie qu'un même code Terraform est utilisé pour déployer aussi bien l'environnement de Non-Production (NP) que celui de Production (PR), la différenciation se faisant uniquement par l'injection de variables de configuration.

Cette stratégie garantit :

- L'homogénéité des environnements, réduisant les risques d'erreurs de configuration.
- La réutilisabilité du code pour accélérer les déploiements futurs.
- La sécurité intégrée (*Security by Design*), chaque module embarquant nativement les standards de durcissement (chiffrement, logs, moindres privilèges).

Les sous-sections suivantes décrivent le périmètre, les choix techniques et les mécanismes de sécurité de chaque module.

10.1 Module – Comptes & Security

Baseline

But

Fournir un socle d'organisation et de sécurité commun pour tous les environnements : création normalisée des comptes AWS (shared-services, nonprod, prod), activation des garde-fous globaux (SCP, journaux, détection), et bootstrap IaC/CI sécurisé pour les déploiements.

Périmètre

- AWS Organizations (unité racine + OUs **shared**, **nonprod**, **prod**)
- Création/"vending" des comptes : **shared-services**, **np**, **pr**
- IAM Identity Center (SSO) configuré, MFA obligatoire
- Service Control Policies (SCP) organisationnelles (EU-only, deny S3 public, audit non désactivable)
- CloudTrail org-wide, Config, Security Hub, GuardDuty activés et centralisés
- Buckets de logs/audit (S3, Object Lock optionnel)
- Budgets/Cost Anomaly Detection et tagging enforced (Project, Env, Owner, CostCenter)
- Bootstrap Terraform + intégration OIDC (pipeline infra)

Variabilisation

- **ou_structure** : arborescence des OUs à créer/valider
- **accounts** : liste des comptes à "vendre" (nom, e-mail, OU)
- **scp_sets** : jeux de politiques à attacher par OU/compte (eu_only, no_public_s3, mandatory_trail, ...)

Enzo Loppin-Antimi - Studi Promotion KAO

- **security_services** : activation/paramètres (CloudTrail, Config, GuardDuty, Security Hub)
- **central_log_bucket** : nom, région, object lock (on/off)
- **sso_groups_roles** : mapping groupes SSO → rôles (Admin, PowerUser, ReadOnly, CICD)
- **oidc_trusts** : référentiels d'identités CI/CD autorisés (GitLab/GitHub)
- **budget_thresholds** : seuils d'alerte coûts par compte/étiquette

Sécurité

- SSO + MFA obligatoires, aucun utilisateur IAM persistant
- SCP de déni sur actions risquées (désactivation de CloudTrail/Config, création S3 public, régions non autorisées)
- Permissions minimales via boundaries et rôles dédiés (Admin/PowerUser/ReadOnly/CI/CD)
- Journaux centralisés et immuables (S3 Object Lock si activé)
- Intégration CI/CD par OIDC (pas de clés statiques)
-

Observabilité

- CloudTrail org vers bucket central, CloudWatch métriques d'audit
- Security Hub agrégé : conformité Foundational/CIS, findings GuardDuty corrélés
- Tableaux de bord coûts/usage (budgets, anomalies)
- Visibilité Config (drifts et ressources non conformes)

Livrables

- Module Terraform agnostique **accounts-baseline** (org, OUs, comptes, SCP, SSO, guardrails)
- Dossiers/rôles SSO et mappages groupes → rôles documentés
- Buckets de logs + politiques (versioning, retention, object lock)
- Pipelines d'amorçage infra (OIDC, stages validate/plan/scan/apply)
- Runbook "Account vending" (création/modif/suspension) et "Incident d'audit" (investigation)

Valeur

- Fondation gouvernée et reproductible pour tous les environnements
- Réduction du risque par garde-fous organisationnels et journaux centraux
- Accès unifiés et éphémères (SSO/OIDC), auditabilité bout-en-bout
- Prévisibilité budgétaire et contrôle des dérives

Bonnes pratiques

- Toujours attacher un **SCP 'mandatory-trail'** à la racine pour empêcher toute désactivation d'audit
- Appliquer **EU-only** au niveau OU pour éviter des déploiements hors zone
- Créer les comptes via laC (et non console) pour garder la traçabilité (change set Terraform)
- Forcer **Block Public Access** sur tout bucket de journaux et interdire **s3:PutBucketAcl** public via SCP
- Tester l'accès CI/CD par OIDC sur un compte sandbox avant d'autoriser NP/PR
- Vérifier périodiquement la **couverture GuardDuty/Security Hub/Config** et corriger les comptes orphelins

Service Control Policies (3 essentielles)

SCP 1 : Restriction régions EU uniquement

Scope : Toute l'organization (tous comptes, toutes OUs)

But : Conformité RGPD – aucune ressource AWS hors Europe

Fonctionnement : Refuse actions AWS hors eu-west-1, eu-central-1, eu-west-2. Services globaux (IAM, CloudFront) exemptés.

Cas concret : Dev tente créer instance us-east-1 → SCP refuse → "Access Denied"

Déploiement : Terraform + AWS Organizations

Variabilisation : Aucune (immuable, toute l'organisation)

SCP 2 : Blocage S3 publics

Scope : Toute l'organization

But : Zéro exposition données personnels

Fonctionnement :

- Refuse désactivation "Block Public Access"
- Refuse ACL public sur buckets
- Refuse permissions S3 publiques via policy

Cas concret : Admin tente rendre bucket public → SCP refuse

Déploiement : Terraform + AWS Organizations

Variabilisation : Aucune (immuable, toute l'organisation)

SCP 3 : CloudTrail non-désactivable

Scope : Toute l'organization

But : Audit immuable, impossible "éteindre les lumières"

Fonctionnement : Refuse :

- StopLogging CloudTrail
- DeleteTrail CloudTrail
- Modifications config CloudTrail
- Suppression bucket CloudTrail

Cas concret : Admin compromis tente arrêter CloudTrail → SCP refuse → logs continuent

10 .2 Module – Core Network

Déploie l'infrastructure réseau AWS de RHorizon: VPC segmenté, routage contrôlé et endpoints privés, afin de garantir l'isolation des flux et la connectivité sécurisée vers les services managées d'AWS

Périmètre

- VPC dédié (CIDR, DNS activé)
- Subnets publics / applicatifs / data
- Internet Gateway & NAT Gateway(s)
- Tables de routage par tier
- VPC Endpoints privés (S3, STS, ECR, Logs, Secrets, KMS)
- Flow Logs + DNS privé (PHZ)

Variabilisation

- **az_count** : nombre de zones de disponibilité
- **nat_strategy** : NAT unique ou une par AZ
- **endpoints** : services AWS accessibles en connection privé
- **flow_logs** : destination des logs (CloudWatch, S3, ou les deux)
- **retention_days** : durée de rétention des logs
- **enable_phz** : activer/désactiver la zone DNS privée

Sécurité

- Security Groups spécifiques par tier (front, app, db)
- Interdiction d'adresses publiques pour les subnets applicatifs et data
- Routage minimaliste : public → IGW, app/data → NAT + endpoints
- Endpoints privés pour éviter l'exposition Internet des services sensibles

Observabilité

- Flow Logs activés sur chaque subnet
- Export configurable vers CloudWatch et/ou S3
- Métriques sur le trafic et anomalies de routage

Livrables

- Module Terraform **agnostique** `core-network`
- Documentation des variables
- Runbook réseau (tests de routage, résolution DNS, vérif logs)

Valeur

- Réseau segmenté et cohérent pour tous les environnements
- Paramétrable (coûts vs haute dispo) via simples variables
- Sécurité et auditabilité intégrées
- Compatible avec les standards RGPD/ANSSI

Bonnes pratiques

- Toujours activer les **Flow Logs** pour chaque subnet.
- Ne jamais autoriser de route directe vers IGW pour les subnets applicatifs et data.
- Vérifier que les **endpoints privés** configurés correspondent aux services utilisés.
- Limiter les règles Security Groups au strict nécessaire (ex. ALB → front uniquement).
- Utiliser des **tags obligatoires** (Project, Env, Owner, CostCenter) pour toutes les ressources

Règles des Security Groups par Tier

1. Tier Public – Application Load Balancer (ALB)

Security Group : `sg-alb`

Direction	Protocole	Port(s)	Source/Destination	Action	Rôle
Ingress	TCP	80	0.0.0.0/0	ACCEPT	Clients HTTP (redirection vers 443)
Ingress	TCP	443	0.0.0.0/0	ACCEPT	Clients HTTPS (Accès public sécurisé)
Egress	TCP	8080-8443	<code>sg-app</code>	ACCEPT	Forwarding : L'ALB envoie le trafic vers les Pods/Nodes EKS

Egress	TCP	443	0.0.0.0/0	ACCEPT	Sortie pour Health Checks de l'ALB et webhooks
---------------	-----	-----	-----------	--------	--

Implications Sécurité **sg-alb** :

- **Deny-by-default** : Tout trafic est **refusé** par défaut
- **Isolation DB** : L'**ALB** n'a pas d'accès direct à la **DB** (port 5432 non listé en Egress).
- **Accès S3** : L'accès nécessite un **VPC Endpoint** et une politique **IAM** pour des raisons de sécurité.

2. Tier Applicatif – EKS Pods / Worker Nodes

Security Group : **sg-app**

Direction	Protocole	Port(s)	Source/Destination	Action	Rôle
Ingress	TCP	8080-8443	sg-alb	ACCEPT	Accepte les requêtes provenant uniquement de l'ALB
Ingress	TCP/UDP	53	\$10.x.x.0/16\$	ACCEPT	DNS interne (Communication avec CoreDNS/Serveur DNS VPC)
Egress	TCP	5432	sg-rds	ACCEPT	DB Access : Les Pods accèdent à PostgreSQL (via le SG RDS)

Egress	TCP	443	0.0.0.0/0	ACCEPT	Appels aux APIs externes (Secrets Manager, KMS, autres services AWS/tiers)
Egress	TCP	53	0.0.0.0/0	ACCEPT	DNS externe (résolution de noms de domaines publics)

Sécurité **sg-app** :

- **Accès Entrant Strict** : Les Pods acceptent les requêtes **uniquement de l'ALB**, garantissant que le tier public agit comme seul point d'entrée.
- **Isolation S3 (Améliorée)** : L'accès à S3 doit utiliser un **VPC Endpoint** combiné à **IRSA (IAM Roles for Service Accounts)** pour des permissions granulaires.
- **Contrôle Inter-Pods** : Pour une isolation plus fine entre les Pods au sein du cluster EKS une **NetworkPolicy Kubernetes** est nécessaire.

3. Tier Data – RDS PostgreSQL

Security Group : `sg-rds`

Direction	Protocole	Port(s)	Source/Destination	Action	Rôle
Ingress	TCP	5432	<code>sg-app</code>	ACCEPT	Accès DB : SEUL le tier Applicatif peut envoyer des requêtes
Egress	Tout	Tout	Tout	Allow	Souvent pour les mises à jour et la communication avec AWS (CloudWatch, etc.).

Sécurité `sg-rds` :

- **Isolation Complète** : RDS est **complètement isolé**. L'accès au port DB est strictement limité à l'**ALB (sg-app)**.
- **Interface Unique** : La seule interface exposée est l'exécution de **requêtes SQL**.
- **Pas d'Accès Accidentel** : Empêche le *copy-paste* d'une règle ouverte, car la source est limitée au SG applicatif.
- **Sécurité Multi-tenant** : L'isolation fine des sources Ingress est cruciale, y compris pour une éventuelle **isolation multi-tenant** future sur les clusters EKS.

Routage Par Tier

Tier (Sous-réseau)	CIDR de Destination	Cible de Routage	Objectif du Trafic
Public Subnets (ALB)	10.x.0.0/16	local	Trafic VPC (interne)
	0.0.0.0/0	IGW (Internet Gateway)	Internet direct (pour l'accès public)
App Subnets (EKS)	10.x.0.0/16	local	Trafic VPC (interne)
	\$0.0.0.0/0	NAT Gateway	Sortant vers Internet (via NAT)
Data Subnets (RDS)	\$10.x.0.0/16	local	Trafic VPC seulement (interne)
	\$0.0.0.0/0	(Non routé)	Pas d'accès à Internet

10 .3 Module - Shared Core (S3 & KMS)

But

Centraliser la journalisation, les sauvegardes et les artefacts applicatifs dans des buckets S3 sécurisés et immuables, protégés par des clés KMS dédiées. Assurer la conformité, la traçabilité et l'intégrité des données.

Périmètre

- Buckets S3 pour logs, backups et assets
- Clés KMS distinctes pour logs, data et assets
- Accès inter-comptes en écriture seule
- Versioning et Object Lock pour la conservation immuable des journaux
- Politiques de cycle de vie (IA, Glacier)

Variabilisation

- `bucket_names` : noms personnalisables pour chaque usage
- `enable_object_lock` : activation de l'immuabilité des journaux
- `kms_keys` : génération de nouvelles clés ou réutilisation de clés existantes
- `lifecycle_days` : durée avant transition vers IA ou Glacier
- `cross_account_roles` : activer/désactiver les autorisations inter-comptes
- `tags_required` : balises obligatoires pour gouvernance

Sécurité

- Chiffrement SSE-KMS appliqué par défaut
- Bloquer tout accès public (Block Public Access activé)
- Permissions en écriture seule pour les comptes consommateurs
- Journalisation des appels API S3 et KMS via CloudTrail

Observabilité

- Logs d'accès activés sur chaque bucket
- Suivi CloudTrail des opérations S3 et KMS
- Tableaux de bord de consommation (volume stocké, requêtes, erreurs)

Livrables

- Module Terraform agnostique `shared-core`
- Documentation d'usage (variables et scénarios)
- Runbook de restauration rapide (S3/KMS)

Valeur

- Un socle unique et réutilisable pour logs, backups et artefacts
- Sécurité renforcée par l'immutabilité et le chiffrement KMS
- Gouvernance simplifiée avec centralisation et tagging
- Conformité RGPD et standards d'audit

Bonnes pratiques

- Toujours activer le versioning sur les buckets S3.
- Vérifier que `enable_object_lock` est activé pour le bucket de logs.
- Appliquer une durée de cycle de vie claire (`lifecycle_days`) pour contrôler les coûts de stockage.

- S'assurer que chaque clé KMS dispose de politiques restrictives (usage limité aux buckets concernés).
- Tester régulièrement la restauration d'un objet supprimé ou remplacé.

10 .4 Module Ingress (ACM, ALB, WAF, DNS)

But

Exposer les applications via un point d'entrée HTTPS sécurisé, hautement disponible et protégé par un WAF. Offrir une gestion centralisée des certificats et du DNS pour simplifier l'accès et renforcer la sécurité.

Périmètre

- Certificats TLS via ACM (validation DNS, renouvellement automatique)
- Load Balancer applicatif (ALB) multi-AZ avec redirection HTTP → HTTPS
- WAFv2 pour filtrage et protection applicative
- Enregistrements DNS publics via Route 53
- Intégration Ingress Kubernetes via AWS Load Balancer Controller

Variabilisation (agnosticité)

- `domain_name` : nom de domaine utilisé
- `enable_waf` : activation ou non de la WebACL
- `waf_rules` : règles managées (OWASP) et/ou personnalisées
- `waf_rate_limit` : seuil de protection contre le trafic abusif

Enzo Loppin-Antimi - Studi Promotion KAO

- `log_destination` : choix de destination des logs (S3, CloudWatch)
- `certificate_validation` : type de validation ACM (DNS par défaut)

Sécurité

- Chiffrement TLS obligatoire (HTTPS only)
- Redirection automatique HTTP → HTTPS
- Application de règles WAF pour bloquer attaques courantes (SQLi, XSS, IP malveillantes)
- Journaux ALB et WAF activés et exportés vers stockage sécurisé

Observabilité

- Logs ALB et WAF centralisés (S3, CloudWatch)
- Tableaux de bord sur le trafic (RPS, erreurs 4xx/5xx, latence)
- Alertes configurées sur dépassement de taux d'erreurs ou de requêtes bloquées

Livrables

- Module Terraform agnostique `ingress-edge`
- Documentation d'usage (variables et schéma DNS/ALB/WAF)
- Runbook de validation HTTPS/WAF

Valeur

- Accès sécurisé et chiffré pour toutes les applications
- Réduction de la surface d'attaque avec WAF et filtrage intelligent
- Traçabilité complète des requêtes et blocages
- Simplification de la gestion des certificats et du DNS

Bonnes pratiques

- Toujours forcer la redirection HTTP → HTTPS.
- Vérifier la validité du certificat ACM avant chaque déploiement.
- Activer systématiquement les logs ALB et WAF.
- Définir une règle de rate-limit adaptée pour limiter les abus sans bloquer le trafic légitime.
- Tester régulièrement que les règles WAF bloquent bien les attaques connues (SQLi, XSS).

10.5 Module – EKS

But

Enzo Loppin-Antimi - Studi Promotion KAO

Déployer une plateforme Kubernetes managée pour exécuter les workloads frontend et backend. Offrir une orchestration scalable, résiliente et sécurisée tout en intégrant les mécanismes natifs d’AWS pour l’authentification et la supervision.

Périmètre

- Cluster EKS (version figée et maintenue)
- Nodegroups dédiés (system, workloads applicatifs)
- IAM Roles for Service Accounts (IRSA)
- RBAC strict pour la gestion des accès Kubernetes
- NetworkPolicies (deny-by-default + règles explicites)
- Add-ons essentiels (metrics-server, AWS LB Controller, autoscaler)

Variabilisation

- `cluster_version` : version Kubernetes à déployer
- `node_instance_types` : types d’instances pour les nodegroups
- `node_scaling` : configuration min/max/desired pour les nodes
- `enable_irsa` : activation de l’accès IAM par service account
- `enable_network_policies` : gestion fine des flux inter-pods
- `addons` : liste des composants installés (metrics-server, autoscaler, ingress controller)

Sécurité

- RBAC strict : pas de cluster-admin global par défaut

Enzo Loppin-Antimi - Studi Promotion KAO

- IRSA activé pour limiter les permissions des pods
- NetworkPolicies appliquées pour restreindre les flux entre namespaces
- Images déployées figées et contrôlées (pas de `:latest`)

Observabilité

- Logs du cluster et des pods envoyés vers CloudWatch
- Métriques collectées via Prometheus et visualisées dans Grafana
- Dashboards dédiés aux workloads et au cluster (latence, saturation, erreurs)

Livrables

- Module Terraform agnostique `eks-basic`
- Documentation d'usage (variables, intégrations IRSA/RBAC)
- Runbook d'exploitation du cluster

Valeur

- Plateforme standardisée et reproductible pour tous les workloads
- Isolation et sécurité natives grâce à IRSA et RBAC
- Scalabilité automatique (pods et nodes)
- Observabilité intégrée pour exploitation et diagnostic

Bonnes pratiques (propres à EKS)

- Toujours figer la version Kubernetes (`cluster_version`) et planifier les mises à jour.
- Activer IRSA pour chaque service nécessitant un accès AWS, plutôt que des clés statiques.
- Interdire les conteneurs root et l'usage de `:latest` via admission policies.
- Déployer systématiquement les add-ons de base (metrics-server, autoscaler, ingress controller).
- Vérifier la présence et l'efficacité des NetworkPolicies dans chaque namespace critique.

10.6 Module – RDS (PostgreSQL)

But

Assurer la persistance des données applicatives via une base PostgreSQL managée sur AWS RDS. Garantir disponibilité, sécurité et conformité, tout en simplifiant les opérations de sauvegarde et de supervision.

Périmètre

- Instance RDS PostgreSQL managée
- `db_subnet_group` dédié (sous-réseaux privés)
- Sauvegardes automatiques (rétention paramétrable)
- Logs intégrés à CloudWatch et/ou exportés S3
- Chiffrement des données (au repos et en transit)
- Security Groups restreints (app → DB uniquement)

Variabilisation

- `instance_class` : taille et type d'instance (ex. `t3.medium`, `m5.large`)

Enzo Loppin-Antimi - Studi Promotion KAO

- `multi_az` : activer ou non la haute disponibilité multi-AZ
- `backup_retention_days` : durée de conservation des sauvegardes
- `enable_pitr` : activer ou non la restauration à un instant précis
- `log_exports` : types de logs activés (audit, erreurs, slow queries)
- `storage_type` : choix du stockage (gp2/gp3/io1)

Sécurité

- Chiffrement SSE-KMS obligatoire au repos
- SSL/TLS forcé pour toutes les connexions
- Credentials gérés par AWS Secrets Manager (rotation activée)
- Accès réseau limité au strict nécessaire (SG applicatif → DB uniquement)

Observabilité

- Intégration des logs RDS avec CloudWatch
- Export optionnel des journaux vers S3 (immutabilité activable)
- Monitoring avancé (Performance Insights activable)
- Tableaux de bord sur l'utilisation (CPU, connexions, stockage)

Livrables

- Module Terraform agnostique `rds-basic`
- Documentation d'usage (variables, schéma de connexion sécurisé)
- Runbook restauration/sauvegarde (snapshot, PITR)

Valeur

- Base de données managée, fiable et conforme
- Paramétrable selon les besoins (coûts vs haute disponibilité)
- Sauvegardes automatisées avec options de restauration avancées

- Supervision intégrée et traçabilité des accès

Bonnes pratiques recommandé

- Toujours activer SSL/TLS et refuser les connexions non chiffrées.
- Vérifier que `backup_retention_days` est supérieur à 0 (sauvegardes jamais désactivées).
- Activer `multi_az` pour les environnements critiques.
- Tester régulièrement la restauration d'un snapshot ou d'un PITR.
- Restreindre les logs activés à ceux utiles (erreurs, requêtes lentes) pour éviter le bruit.

10.7 Module – ECR

Centraliser les images Docker du projet (frontend, backend, outils) dans un registre unique et sécurisé. Garantir l'intégrité des artefacts, la traçabilité des push/pull et le respect de la supply chain logicielle.

Périmètre

- Dépôts ECR pour images applicatives et techniques
- Scan automatique des images à l'import (scan on push)
- Immutabilité des tags pour empêcher l'écrasement
- Chiffrement SSE-KMS appliqué aux artefacts
- Journalisation des actions via CloudTrail

Variabilisation

- `repositories` : liste des dépôts à créer (frontend, backend, ops, etc.)
- `scan_on_push` : activer/désactiver l'analyse de vulnérabilités

Enzo Loppin-Antimi - Studi Promotion KAO

- `tag_immutability` : forcer ou non l'immutabilité des tags
- `kms_key` : clé KMS utilisée pour le chiffrement
- `lifecycle_policy` : stratégie de rétention des images
- `log_destination` : destination des journaux (CloudTrail, S3, CloudWatch)

Sécurité

- Chiffrement SSE-KMS activé par défaut
- Scan automatique des vulnérabilités sur chaque push
- Tags immuables pour préserver l'intégrité des images
- Accès différenciés (push/pull) selon les rôles CI/CD

Observabilité

- Journaux des opérations push/pull via CloudTrail
- Tableaux de bord d'usage (nombre d'images, volume, résultats de scan)
- Alertes sur vulnérabilités critiques détectées

Livrables

- Module Terraform agnostique `shared-ecr`
- Documentation d'usage (variables et politiques de rétention)
- Runbook de gestion des images (push, pull, purge)

Valeur

- Registre unique et gouverné pour toutes les équipes
- Intégrité et conformité de la supply chain logicielle

- Réduction des coûts via lifecycle policy
- Traçabilité complète des push/pull et audits facilités

Bonnes pratiques

- Toujours activer `scan_on_push` pour chaque dépôt.
- Forcer `tag_immutability = true` pour bloquer les réécritures.
- Définir une `lifecycle_policy` claire (ex. conserver 10 dernières images).
- Ne jamais utiliser de tag générique (`latest`) dans les dépôts critiques.
- Vérifier régulièrement les journaux CloudTrail pour auditer les accès.

10.8 Module CI/CD Infra

But

Automatiser le provisionnement et la gestion de l'infrastructure via des pipelines CI/CD sécurisés. Garantir la traçabilité des "plans", l'application contrôlée des changements et l'intégration des règles de sécurité dès le déploiement.

Périmètre

- Pipelines GitLab CI/CD pour l'infrastructure
- Intégration OIDC avec AWS (rôles éphémères, pas de clés statiques)
- Contrôles de sécurité automatisés (lint, checkov, OPA/Conftest)
- Conservation des artefacts d'audit (plans, rapports de scan)

Variabilisation

Enzo Loppin-Antimi - Studi Promotion KAO

- **opa_policy_set** : règles de conformité à appliquer (EU-only, WAF obligatoire, pas de S3 public)
- **scan_levels** : seuils de gravité pour bloquer ou non un déploiement (warning, critical)
- **artifact_retention** : durée de conservation des plans et rapports
- **environments** : mapping NP/PR pour les jobs (auto vs gated)
- **regions_allowed** : liste des régions autorisées pour le déploiement

Sécurité

- Exécution des pipelines avec runners éphémères
- Rôles OIDC en least privilege, rotation automatique des jetons
- Policy-as-Code pour refuser tout plan non conforme
- Signature et conservation des artefacts critiques (plans, SBOM, attestations)

Observabilité

- Logs de pipelines stockés dans GitLab et exportables pour audit
- Tableaux de bord CI/CD (taux de succès, temps d'exécution, nombre de refus de policy)
- Alertes sur échec de jobs critiques (plan, apply, scan sécurité)

Livrables

- Module Terraform agnostique **cicd-infra**
- Pipelines GitLab CI/CD préconfigurés (stages : validate, plan, scan, apply)
- Documentation d'usage (règles OIDC, workflows GitLab)
- Runbook CI/CD (rollback, gestion d'erreurs de déploiement)

Valeur

- Infrastructure déployée de manière reproductible et sécurisée
- Conformité intégrée grâce aux contrôles automatiques
- Traçabilité complète des actions et changements
- Réduction du risque humain grâce à l'automatisation et aux gates

Bonnes pratiques (propres au CI/CD Infra)

- Toujours exécuter `terraform validate` et `checkov` avant tout plan.
- Refuser automatiquement tout plan qui ne respecte pas les règles OPA critiques.
- Ne conserver que les artefacts nécessaires (plans validés, rapports finaux), purger le reste pour limiter l'exposition.
- Activer systématiquement l'exécution dans des runners éphémères localisés en UE.
- Documenter chaque gate manuel (prod) avec justification et horodatage.

10.9 Module - Secrets Management

But

Fournir un socle de gestion des secrets centralisé, chiffré et auditable, consommable par les applications (EKS via CSI Driver) et par les pipelines CI/CD, sans exposition en clair ni circulation de clés statiques.

Périmètre

- Création des coffres de secrets (AWS Secrets Manager).
- Politiques d'accès fines et restreintes par rôle (IRSA pour EKS, rôles CI/CD).
- Intégration Kubernetes via Secret Store CSI Driver (manifests/Helm).

Enzo Loppin-Antimi - Studi Promotion KAO

- Rotation automatique des secrets (moteur managé ou Lambda).
- Chiffrement avec KMS (clé dédiée ou clé partagée).

Variabilisation

- `kms_key_arn` : clé KMS utilisée (externe ou créée ailleurs).
- `secrets_catalog` : liste de secrets à créer (nom, description, labels, rotation on/off, period).
- `access_bindings` : mapping secret → rôles autorisés (ARNs IRSA, rôles CI/CD).
- `rotation_engine` : `managed` | `lambda` | `none`.
- `recovery_window_days` : fenêtre de récupération avant suppression définitive.
- `enable_csi` : installation/activation du driver CSI pour EKS.
- `tags_required` : balises de gouvernance (Project, Env, Owner, CostCenter).

Sécurité

- Chiffrement SSE-KMS pour tous les secrets.
- Resource policies restrictives (principals explicites, option `aws:PrincipalOrgID`).
- Accès applicatif via IRSA (pas de clés statiques dans les pod ni en pipeline).
- Pas de valeur de secret en clair dans Terraform (secret sensibles injectés par pipeline).

Observabilité

- Journalisation des accès/rotations via CloudTrail.
- Métriques/alertes sur échecs de rotation (CloudWatch Alarms).
- Traces d'usage côté appli via annotations CSI (facilitent le troubleshooting).

Livrables

- Module Terraform agnostique `secrets-management`.
- Manifests/Helm pour Secret Store CSI Driver et `SecretProviderClass`.
- Documentation d'usage (modèle `secrets_catalog`, exemples d'accès IRSA).
- Runbook : création/mise à jour/rotation, procédure de retrait d'accès.

Valeur

- Élimination des secrets en clair (CI/CD et runtime).
- Contrôle d'accès minimal et traçable par rôle.
- Rotation automatisée et vérifiable.
- Intégration native avec EKS et pipelines.

Bonnes pratiques

- Ne jamais définir la **valeur** d'un secret en dur dans Terraform : créer le conteneur (`aws_secretsmanager_secret`) et injecter la version via variable sensible du pipeline (`aws_secretsmanager_secret_version`), puis masquer les outputs.
- Lier chaque secret à des **principaux explicites** (IRSA/pipelines) dans `access_bindings` ; refuser toute wildcard.
- Activer la **rotation** pour les secrets à longue durée de vie (DB creds, API keys), et alerter sur tout échec de rotation.
- Fixer un `recovery_window_days` (>0) pour éviter les suppressions irréversibles accidentelles.

11. Configuration des Environnements

11.1 — Module Comptes & Security Baseline

Nonprod

scp_sets :

Nous appliquons uniquement le set “basic”, ce qui permet de conserver les protections essentielles — telles que la restriction aux régions européennes, le blocage des buckets publics ou l’obligation d’activer CloudTrail — tout en laissant suffisamment de flexibilité aux développeurs pour itérer rapidement et tester leurs fonctionnalités sans contraintes excessives.

cloudtrail_object_lock :

Désactivé, car la priorité de Nonprod est d’offrir un cycle de debugging rapide et adapté au développement. Comme les logs n’ont pas vocation à constituer une preuve légale ou une archive d’audit, l’absence d’immutabilité permet de réduire légèrement les coûts tout en restant cohérente avec un environnement non sensible.

cloudtrail_retention_days :

Configuré à 14 jours, ce qui représente une durée adéquate pour couvrir les cycles de test et permettre aux développeurs de revenir sur les événements récents en cas de problème, sans conserver inutilement des données obsolètes.

iam_mfa_enforcement :

Optionnel, afin d’éviter d’interrompre trop fréquemment les développeurs dans leurs tâches quotidiennes. Cette flexibilité est acceptable car l’environnement ne contient aucune donnée critique nécessitant un niveau de protection élevé.

iam_session_duration_hours :

Définie à 12 heures, ce qui permet à un utilisateur de travailler toute la journée sans devoir renouveler sa session. Cette configuration accélère les workflows en limitant les reconnections répétitives, tout en gardant un risque maîtrisé dans un environnement non sensible.

Production

scp_sets :

Nous activons le set “basic” complété par le set “security”, ce qui renforce significativement les contrôles d’accès et les restrictions possibles. Cette approche est parfaitement adaptée à la Production, où les coûts légèrement plus élevés sont justifiés par la nécessité d’un niveau de sécurité strict et d’une gouvernance alignée RGPD et ANSSI.

cloudtrail_object_lock :

Activé pour une durée de 90 jours, garantissant que l’intégralité des logs CloudTrail

est stockée de manière immuable. Ce mécanisme apporte une valeur légale importante, car même un administrateur compromis ne pourrait pas altérer ou supprimer les journaux, assurant ainsi une traçabilité totale en cas d'incident ou d'audit.

cloudtrail_retention_days :

Fixé à 90 jours, ce qui correspond au minimum attendu dans le cadre des exigences de conformité. Cette durée garantit que toute enquête, même déclenchée plusieurs semaines après un événement, puisse s'appuyer sur des logs complets et vérifiables.

iam_mfa_enforcement :

Obligatoire pour tous les utilisateurs, sans aucune exception, afin de protéger efficacement l'accès aux données RH sensibles. Même en cas de compromission d'un mot de passe, la deuxième étape d'authentification bloque toute tentative d'accès non autorisé.

iam_session_duration_hours :

Réduite volontairement à 1 heure, ce qui diminue drastiquement la fenêtre d'opportunité pour un attaquant qui aurait réussi à accéder à un compte légitime. Cette durée courte représente un compromis optimal entre sécurité maximale et ergonomie.

11.2 — Module Core Network

Nonprod

az_count :

L'environnement Nonprod est déployé sur **2 zones de disponibilité**, ce qui offre un niveau de résilience suffisant pour du développement tout en limitant les coûts. Une perte d'AZ entraînerait un downtime temporaire, mais cette contrainte reste acceptable dans un contexte non critique où aucune charge client n'est en production.

vpc_cidr :

La plage réseau **10.10.0.0/16** est utilisée afin d'isoler clairement Nonprod et de permettre une croissance progressive. Ce format généreux garantit qu'il sera toujours possible d'ajouter des sous-réseaux supplémentaires sans procéder à une refonte du schéma réseau.

subnet_cidr_mask :

Les sous-réseaux sont définis en **/24**, ce qui offre environ 251 adresses IP utilisables par subnet. Même si ce chiffre peut être légèrement serré dans certains scénarios de développement intensif, il reste tout à fait suffisant pour un environnement dont le scaling maximal est maîtrisé et limité.

nat_strategy :

Une seule **NAT Gateway** est utilisée pour réduire les coûts. Cette configuration introduit un point de défaillance potentiel, mais celui-ci est considéré comme acceptable dans un environnement non critique où les développeurs peuvent relancer manuellement leurs tests en cas d'indisponibilité.

vpc_endpoints :

Seuls les endpoints essentiels — notamment S3, STS et ECR — sont activés. Ce choix permet d'équilibrer correctement les coûts tout en évitant les frais de transfert réseau associés à l'utilisation publique de ces services.

flow_logs_retention_days :

Les Flow Logs sont conservés pendant **14 jours**, une durée cohérente avec les besoins de debugging court-terme. Cela permet de retracer les communications réseau récentes sans générer un volume de stockage excessif.

Production

az_count :

L'environnement Production s'appuie sur **3 zones de disponibilité** afin d'assurer un SLA de 99.9 %, indispensable à un service RH accessible par des utilisateurs finaux. Cette architecture multi-AZ permet un basculement quasi instantané vers une zone saine en cas de panne, sans impact visible pour l'utilisateur.

vpc_cidr :

Nous utilisons la plage **10.20.0.0/16**, totalement distincte de Nonprod, ce qui évite toute collision de routage et facilite la gestion réseau à long terme grâce à une séparation logique nette.

subnet_cidr_mask :

Les subnets en Production sont dimensionnés en **/23**, offrant 512 adresses IP, ce qui constitue une marge de capacité indispensable pour supporter des montées en charge importantes, du scaling automatique et la présence de multiples composants comme des pods, des nœuds EKS ou des services managés.

nat_strategy :

Chaque AZ dispose de sa **NAT Gateway dédiée**, garantissant l'absence de bottleneck et une haute disponibilité des flux sortants. Cette architecture assure qu'un pod situé dans une AZ ne traversera jamais une autre zone pour sortir sur Internet, ce qui optimise la latence et la résilience.

vpc_endpoints :

L'ensemble des endpoints critiques est activé : S3, ECR (API et Docker), Secrets Manager, KMS, STS et CloudWatch Logs. Cela réduit drastiquement les coûts liés aux appels publics, et représente un gain financier notable dès lors que le volume d'images conteneurs ou de logs devient conséquent.

flow_logs_retention_days :

Les logs sont conservés **90 jours** et stockés sur un bucket S3 immuable (Object Lock). Cette configuration permet de garantir une observabilité optimale du trafic réseau et une capacité d'investigation complète en cas d'incident.

11.3 — Module Ingress (ALB, TLS, WAF, DNS)

Nonprod

certificate_type :

Nous utilisons un certificat **auto-signé**, ce qui accélère considérablement le déploiement et permet de tester rapidement les flux HTTPS sans engager de procédure de validation publique.

waf_rules_set :

Le WAF applique un jeu de règles **minimal**, centré sur les attaques courantes (injection SQL, XSS, traversée de répertoires). Cette protection est suffisante pour du développement, où l'objectif principal est de tester des fonctionnalités internes plutôt que de résister à des attaques externes.

waf_rate_limit_threshold :

Le seuil est fixé à **5000 requêtes par 5 minutes**, permettant aux développeurs d'effectuer des tests de charge sans être bloqués par le WAF. Cette marge facilite l'itération rapide et l'analyse des comportements sous différentes charges.

route53_failover :

La configuration DNS utilise un **simple alias statique**, sans mécanisme de failover automatisé. En cas de problème, une intervention manuelle peut être menée sans impact important, puisque l'environnement n'est pas exposé à des utilisateurs finaux.

Production

certificate_type :

Un certificat **AWS-managed public** est utilisé. Il est automatiquement renouvelé 90 jours avant expiration, garantissant une continuité de service complète et la confiance des navigateurs clients.

waf_rules_set :

La configuration du WAF est **complète** et inclut les règles managées AWS ainsi que la

threat intelligence, permettant de bloquer automatiquement les IP malveillantes connues et d'offrir une protection renforcée contre des attaques sophistiquées.

waf_rate_limit_threshold :

Le seuil est fixé à **2000 requêtes par 5 minutes**, ce qui rend beaucoup plus difficile toute tentative de brute-force ou attaque volumétrique ciblant les comptes utilisateurs. Cette limite protège efficacement les interfaces exposées.

route53_failover :

Le DNS fonctionne en **active-active multi-AZ** avec health checks toutes les 30 secondes. En cas de défaillance du load balancer d'une AZ, la bascule s'effectue en 1 à 5 minutes selon le cache DNS, assurant une continuité quasiment transparente pour les utilisateurs.

11.4 — Module EKS

Nonprod

instance_type :

Les nœuds EKS utilisent des **t3.medium**, qui sont des instances burstables. Elles permettent de maintenir des coûts très bas tout en offrant des performances suffisantes pour les tests, avec une latence plus élevée en cas de forte charge, ce qui reste acceptable en développement.

hpa_threshold_cpu :

Le seuil est fixé à **70 %**, ce qui rend le scaling un peu moins réactif mais permet d'observer la charge et les comportements applicatifs sur une plage plus large, ce qui est utile pour analyser le fonctionnement de l'application.

network_policies_enforcement :

Les NetworkPolicies ne sont **pas obligatoires**, afin de faciliter le debugging en permettant une communication plus libre entre les pods. L'objectif est de simplifier les expérimentations et les tests.

node_scaling_min_max :

Entre **2 et 4 nœuds**, ce qui constitue un dimensionnement économique tout en permettant d'héberger des charges modérées et d'exécuter plusieurs pods simultanément.

Production

instance_type :

La Production s'appuie sur des **m5.large**, qui offrent une performance CPU stable et non-burstée, garantissant une latence plus prévisible. Ce choix s'impose pour un environnement soumis à des engagements de disponibilité élevés.

hpa_threshold_cpu :

Le HPA est réglé à **60 %**, ce qui le rend plus réactif. Ainsi, lors d'un pic de trafic, l'augmentation du nombre de pods se fait plus rapidement, contribuant au respect du SLA de 99.9 %.

network_policies_enforcement :

L'application des règles est **obligatoire**, avec une stratégie deny-by-default. Cela signifie qu'un pod ne peut communiquer qu'avec les destinations explicitement autorisées, ajoutant une couche de segmentation réseau indispensable en Production.

node_scaling_min_max :

Réglé entre **3 et 10 nœuds**, permettant d'assurer la disponibilité même en cas de panne d'une AZ. Chaque AZ reçoit au moins un nœud, garantissant une capacité de reprise immédiate.

11.5 — Module RDS

Nonprod

multi_az_enabled :

Désactivé, ce qui limite les coûts et réduit la complexité. Une panne du nœud RDS nécessiterait une restauration manuelle, mais cela reste acceptable puisque les données Nonprod ne sont ni critiques ni uniques.

backup_retention_days :

Les backups sont conservés **7 jours**, durée cohérente avec les cycles courts de développement où les données peuvent être régénérées.

pitr_enabled :

Désactivé, car le besoin de restaurer à un instant précis n'est pas nécessaire dans cet environnement.

log_retention_days :

Conservés **5 jours**, ce qui est suffisant pour analyser les erreurs récentes ou les requêtes lentes.

Production

multi_az_enabled :

Activé avec une réplication synchrone, ce qui offre un **RPO de 0 seconde** et un **RTO d'environ 60 secondes**. En cas de panne de l'instance principale, la bascule automatique vers l'AZ secondaire assure une continuité quasi immédiate.

backup_retention_days :

Fixée à **30 jours**, conformément aux bonnes pratiques du RGPD et permettant une analyse rétroactive sur plusieurs semaines.

pitr_enabled :

Activé, offrant la possibilité de restaurer la base à un instant précis dans les 30 derniers jours — un atout majeur en cas d'erreur applicative.

log_retention_days :

Fixés à **90 jours**, avec exportation vers S3 immuable pour garantir une traçabilité totale.

11.6 — Shared Services

s3_buckets :

Trois buckets centralisés sont dédiés aux logs, aux backups et aux artefacts CI/CD. Cette centralisation facilite la gouvernance globale et évite la duplication des ressources entre environnements.

s3_versioning :

Activé partout afin de permettre la restauration rapide d'un fichier accidentellement supprimé ou modifié.

kms_keys_count :

Une clé principale est utilisée actuellement pour chiffrer l'ensemble des données. L'architecture permet l'introduction d'une clé supplémentaire dédiée aux données RH sensibles si cela devient nécessaire.

kms_key_rotation_days :

Rotation automatique toutes les **90 jours**, réduisant le risque lié à l'utilisation prolongée d'une même clé.

object_lock_governance :

Activé pour **90 jours**, assurant l'immuabilité des logs critiques.

route53_zones :

L'organisation utilise le domaine principal *rh horizon.xyz*, ainsi que les sous-domaines *np.rh horizon.xyz* et *pr.rh horizon.xyz*, ce qui garantit une séparation nette et une gestion DNS simplifiée.

cloudtrail_central :

Tous les comptes remontent leurs logs vers un bucket S3 immuable centralisé, garantissant un audit global cohérent.

ecr_registry :

Un registre unique est utilisé, ce qui facilite le partage d'images entre Nonprod et Production tout en simplifiant la gestion des versions.

12 Déploiement continue (CI/CD + conteneurisation)

L'un des principes fondamentaux du projet RHZORION est de garantir une **chaîne de déploiement continue fiable, sécurisée et traçable**. Cela permet d'intégrer rapidement de nouvelles fonctionnalités, de corriger des bugs ou de déployer des correctifs de sécurité, tout en limitant les risques pour l'environnement de production. Cette chaîne repose sur trois piliers : les Dockerfiles, les Charts Helm et les pipelines CI/CD.

12.1 Dockerfiles (Backend Flask et Frontend Vue.js)

Les **Dockerfiles** définissent la recette de construction des images applicatives. Ils sont conçus pour être :

- **stables** (image de base figée par digest SHA256, pas de **latest**),
- **sécurisés** (utilisateur non-root, dépendances verrouillées),
- **reproductibles** (multi-stage builds, suppression des outils de build en étape finale).

Backend Flask

Le Dockerfile du backend se compose de deux étapes :

1. **Build** : installation des dépendances Python à partir d'un **requirements.txt** verrouillé (hashes).
2. **Runtime** : copie minimale du code applicatif dans une image légère, exécutée via Gunicorn, sous un utilisateur applicatif dédié.

Frontend Vue.js

Le Dockerfile du frontend suit un schéma multi-stage :

1. **Build** : compilation des assets Vue.js avec Node.js.

2. **Runtime** : déploiement des fichiers statiques dans une image Nginx optimisée, configurée pour servir l'application.

Les Dockerfiles n'embarquent aucune variable d'environnement sensible. La configuration (URLs, secrets, paramètres applicatifs) est injectée dynamiquement par Kubernetes via ConfigMaps et Secrets Manager.

Chaque image construite est :

- **poussée dans ECR** (cf. module 10.ECR),
- **scannée** automatiquement (Trivy/ECR Scan),
- **taguée** avec un identifiant unique (**version applicative + commit SHA**),
- **signée** via Cosign pour assurer l'intégrité.

12.2 Stratégie de Déploiement Helm (Standardisation)

Pour garantir des déploiements reproductibles et cohérents entre les environnements Non-Prod et Prod, RHZORION utilise **Helm**, le gestionnaire de paquets pour Kubernetes. Plutôt que de gérer des manifestes YAML statiques et dupliqués, nous avons créé des **Charts génériques** (templates) pour le Frontend et le Backend.

Cette approche permet de "templatiser" l'application : le code structurel (Deployment, Service, Ingress) reste identique, seule la configuration (valeurs) change selon l'environnement

1. Structure du Chart Applicatif

Chaque micro-service (Frontend Vue.js, Backend Flask) est empaqueté selon une structure commune :

- **templates/deployment.yaml** : Définit le cycle de vie des Pods (répliques, stratégie de mise à jour).
- **templates/service.yaml** : Expose les Pods en interne via une ClusterIP stable.
- **templates/ingress.yaml** : Configure les règles de routage externe (ALB) et le TLS.
- **templates/hpa.yaml** : Définit les règles d'auto scaling horizontal basées sur l'utilisation CPU/RAM
- **templates/serviceaccount.yaml** : Crée l'identité du Pod pour l'intégration IRSA (accès AWS)
- **values.yaml** : Contient les valeurs par défaut (souvent celles de Non-Prod).

2. Gestion des Environnements (Values Files)

Enzo Loppin-Antimi - Studi Promotion KAO

La distinction entre les environnements ne se fait pas au niveau du code, mais via des fichiers de surcharge spécifiques passés au moment du déploiement (`helm upgrade -f values.pr.yaml`)

Paramètre	values.np.yaml (Non-Prod)	values.pr.yaml (Production)	Justification
replicaCount	1 (ou 2)	min: 3, max: 10 (HPA)	Haute disponibilité et tolérance aux pannes en Prod.
resources	requests: 100m/128Mi	requests: 500m/1Gi	Garantie de performance pour les utilisateurs réels.
ingress.host	np.rhorizon.xyz	app.rhorizon.xyz	Séparation stricte des points d'entrée DNS
logLevel	DEBUG	INFO ou WARN	Verboosité réduite en Prod pour limiter le bruit et le coût de stockage.

3. Zoom sur les Composants Critiques

Les Charts intègrent nativement les bonnes pratiques de fiabilité et de sécurité :

- **Sondes de Santé (Probes) :**
 - *Liveness Probe* : Vérifie si l'application est "vivante". Si elle échoue (ex: deadlock), Kubernetes redémarre le pod
 - *Readiness Probe* : Vérifie si l'application est prête à recevoir du trafic (ex: connexion DB établie). Tant qu'elle échoue, le pod est retiré du Load Balancer
- Sécurité du Contexte (SecurityContext) :
Conformément à la stratégie de sécurité, les conteneurs sont configurés pour s'exécuter avec un `runAsNonRoot: true` et `readOnlyRootFilesystem: true` (si possible) pour empêcher toute modification du système de fichiers en cas d'intrusion
- Injection de Secrets (CSI Driver) :
Les Charts ne contiennent aucun secret. Ils référencent un `SecretProviderClass` qui monte les secrets (DB credentials, API keys) directement depuis AWS Secrets

L'utilisation de Helm permet à RHZORION de :

1. **Standardiser** : Tous les micro-services partagent la même structure de déploiement.
2. **Sécuriser** : Les correctifs de sécurité (ex: ajout d'une SecurityPolicy) sont appliqués dans le Chart et propagés à tous les services lors de la mise à jour.
3. **Auditer** : L'historique des versions (`helm history`) permet de savoir exactement quelle version du Chart est déployée et de faire un retour arrière (`helm rollback`) immédiat en cas de problème.

12.3 Pipelines CI/CD applicatifs (GitLab)

Les pipelines GitLab orchestrent la supply chain logicielle du projet. Ils sont divisés en **stages** distincts, chacun ayant une fonction précise.

Structure de la pipeline ci/cd de notre application

1. Build

- Compilation du frontend et backend.
- Construction des images Docker à partir des Dockerfiles.
- Production d'artefacts (binaires, images).

2. Scan

- Analyse SAST (Static Application Security Testing) pour le code.
- Scan des dépendances (pip/npm audit, Trivy pour images).
- Vérification IaC (Checkov, OPA/Conftest).
- Génération de rapports de vulnérabilités.

3. Signature & publication

- Génération d'un SBOM (Software Bill of Materials) avec Syft.
- Signature cryptographique des images avec Cosign.

- Push des images validées vers ECR.

4. Deploy

- En NP : déploiement automatique en rolling update.
- En PR : gate manuel obligatoire (double approbation).
- Déploiement progressif (canary/blue-green).
- Rollback automatisé si les SLO ne sont pas respectés (latence, erreurs).

Points clés de sécurité

- Intégration OIDC : les pipelines assument des rôles AWS temporaires, pas de clés statiques.
- Gates : un déploiement en PR est bloqué si une vulnérabilité critique est détectée.
- Journalisation : tous les jobs et résultats de scan sont archivés (90j en PR).

12.4 Supply chain sécurisée

La pipeline CI/CD est conçue pour garantir une **traçabilité de bout en bout** :

- **Dockerfiles** → images reproductibles et sécurisées.
- **Scans automatisés** → détection précoce des vulnérabilités.
- **Signatures** (Cosign) → intégrité des artefacts.
- **SBOM** (Syft) → visibilité complète sur les dépendances.
- **Pipelines audités** → logs stockés et accessibles pour contrôle.

Cette supply chain répond aux recommandations ANSSI DevSecOps : séparation NP/PR, signatures des artefacts, scans de sécurité intégrés, et contrôle d'accès strict aux dépôts et aux pipelines.

12. Observabilité & exploitabilité

La supervision et l'exploitabilité sont au cœur du projet RHZORION. Une application SaaS RH doit garantir non seulement sa disponibilité, mais aussi sa performance et sa conformité réglementaire. Pour répondre à ces enjeux, une stack complète d'observabilité a été mise en place, reposant sur quatre piliers : la collecte de métriques, la visualisation, la centralisation des logs et le traçage des requêtes.

12.1 Prometheus – collecte et stockage des métriques

Prometheus est déployé sur EKS comme système central de collecte.

Il agrège :

- **métriques systèmes** : CPU, mémoire, saturation des nœuds EKS, usage disque RDS, trafic réseau,
- **métriques applicatives** : nombre de requêtes HTTP, temps de réponse, erreurs 4xx/5xx,
- **métriques middleware** : files d'attente, nombre de pods actifs, latence inter-services.

Les métriques sont scrappées via exporters standards (Node Exporter, kube-state-metrics) et des endpoints applicatifs `/metrics` instrumentés dans le backend Flask.

En environnement NP, Prometheus fonctionne en instance unique (sans redondance). En PR, un cluster HA est mis en place avec rétention longue des données.

12.2 Grafana – visualisation et tableaux de bord

Grafana sert d'interface unique pour les équipes techniques et métiers.

Des dashboards spécifiques ont été conçus pour couvrir différents besoins :

- **Dashboard applicatif** : latence moyenne, taux d'erreurs HTTP, disponibilité du frontend et backend, temps de réponse des APIs clés (login, export, génération de documents).
- **Dashboard base de données** : connexions actives, requêtes lentes, locks, consommation CPU/mémoire du RDS, volumétrie des tables.
- **Dashboard cluster Kubernetes** : état des pods et des nodes, HPA (autoscaling), saturation mémoire, évictions.

- **Dashboard réseau & sécurité** : trafic entrant ALB, nombre de requêtes bloquées par le WAF, distribution par IP/source géographique.
- **Dashboard utilisateur (métier)** : taux d'authentification réussie, nombre de sessions actives, temps de génération des rapports RH.

Les dashboards sont construits à partir de modèles ([provisioning Grafana](#)) pour garantir la reproductibilité. En NP, les dashboards se limitent aux métriques essentielles, tandis qu'en PR ils couvrent l'ensemble de la chaîne applicative et intègrent des SLO formels (ex. latence < 300 ms sur 95 % des requêtes).

12.3 Fluent Bit – collecte et centralisation des logs

Fluent Bit est déployé en DaemonSet sur chaque nœud Kubernetes pour collecter les logs applicatifs (stdout/stderr) et les envoyer vers :

- **CloudWatch** pour un suivi temps réel,
- **S3** pour archivage long terme (immutabilité via Object Lock en PR).

Les logs sont enrichis avec des métadonnées (namespace, pod, container, commit Git de l'image) pour faciliter le debug et l'audit.

Exemples de scénarios suivis :

- erreurs applicatives critiques ([level=ERROR](#)),
- crashes de pods ou redémarrages en boucle,
- anomalies de trafic (pics d'appels API inhabituels).

12.4 OpenTelemetry – traçage distribué

OpenTelemetry (OTel) est utilisé pour instrumenter le backend et tracer les requêtes de bout en bout.

Cela permet de suivre :

- le chemin complet d'une requête utilisateur (frontend → API backend → DB),
- le temps passé dans chaque composant,
- l'impact d'une dépendance externe (API tierce, S3, etc.).

Les traces sont exportées vers Grafana Tempo et corrélées avec les métriques et logs pour donner une vue complète.

12.5 Alertes et SLOs

Des **alertes** sont définies via Prometheus Alertmanager et intégrées à Grafana :

- Erreurs 5xx > 1 % sur 5 minutes,
- Latence p95 > 500 ms,
- Saturation CPU > 80 % sur un node,
- Espace disque RDS < 15 %,
- WAF : plus de 1000 requêtes bloquées/min.

En environnement NP, les alertes sont configurées en mode non bloquant (notifications Slack/Teams uniquement). En PR, elles déclenchent une escalade : alerte Grafana → notification Slack → appel On-Call si non résolue.

Les **SLOs (Service Level Objectives)** sont définis et suivis dans Grafana :

- Disponibilité de l'application > 99,9 %,
- Latence p95 des requêtes API < 300 ms,
- Erreurs 5xx < 0,5 %.

12.6 Exploitabilité

Enzo Loppin-Antimi - Studi Promotion KAO

L'observabilité va de pair avec l'exploitabilité. Les dashboards et alertes ne sont pas seulement des outils de suivi, mais aussi des leviers pour faciliter l'exploitation quotidienne :

- **Runbooks** associés aux alertes critiques (ex. "Que faire si RDS est saturé ?").
- **Scalabilité automatisée** (HPA + Cluster Autoscaler) déclenchée par les métriques Prometheus.
- **Maintenance prédictive** grâce à l'analyse des tendances (augmentation du trafic, consommation CPU).

En résumé, la stack d'observabilité RHorizon repose sur une articulation claire :

- **Prometheus** pour collecter les métriques,
- **Grafana** pour visualiser et formaliser des SLO,
- **Fluent Bit** pour centraliser et archiver les logs,
- **OpenTelemetry** pour tracer les requêtes distribuées.

13. Sauvegardes & PRA

La continuité de service est un enjeu critique pour une application SaaS RH manipulant des données sensibles et soumises au RGPD. Le projet RHorizon intègre dès la conception une stratégie de sauvegarde et de reprise d'activité (PRA), alignée sur les bonnes pratiques ANSSI et AWS Well-Architected.

13.1 Objectifs RPO/RTO

- **RPO (Recovery Point Objective)** : 15 minutes en production, ce qui correspond à la capacité de restaurer les données au plus proche de l'incident.
- **RTO (Recovery Time Objective)** : 1 heure, garantissant un redémarrage rapide de l'application après un sinistre majeur.
- En environnement Nonprod, les objectifs sont plus souples (RPO 24h, RTO 4h), car la priorité est la fonctionnalité et non la continuité stricte.

13.2 Stratégies de sauvegarde

Base de données (RDS PostgreSQL)

- **Sauvegardes automatiques** activées avec une rétention de 30 jours en PR et 7 jours en NP.
- **PITR (Point-In-Time Recovery)** activé en production pour restaurer à un instant précis.
- **Snapshots manuels** réalisés avant chaque mise à jour majeure.
- Export des journaux d'erreurs et de performance vers S3 pour analyse.

Stockage objet (S3)

Enzo Loppin-Antimi - Studi Promotion KAO

- Buckets critiques (backups, logs, artefacts) configurés avec **versioning** et **Object Lock** pour éviter la suppression ou modification accidentelle.
- Politiques de cycle de vie appliquées : archivage dans **Glacier** au-delà de 90 jours.
- Réplication possible vers un bucket secondaire dans une autre région pour PRA cross-region.

Registre conteneurs (ECR)

- Stratégie de rétention via **lifecycle policy** (ex. 10 dernières images conservées).
- Sauvegardes régulières des images critiques (versions stables) exportées vers S3.
- Vérification de l'intégrité via signatures Cosign.

Configuration & IaC

- Tous les scripts Terraform, Helm et CI/CD stockés dans GitLab, sauvegardés automatiquement.
- Snapshots réguliers des dépôts exportés vers un stockage externe sécurisé.

13.3 Scénarios PRA

Plusieurs scénarios ont été modélisés:

1. Perte d'une zone de disponibilité (AZ)

- Basculement automatique sur les ressources restantes (Multi-AZ activé en PR).
- Aucun impact majeur attendu en PR, indisponibilité possible en NP (Single-AZ).

2. Corruption de la base de données

- Utilisation de PITR pour restaurer au dernier état cohérent.
- Vérification via snapshots manuels pour réaliser un rollback si nécessaire.

3. Indisponibilité d'une région

- En PR, option de PRA cross-region (réplication S3 + possibilité de recréer l'infra via Terraform).
- Priorité : restaurer un service minimal en 1h avec données critiques accessibles.

4. **Suppression ou altération de données**

- Object Lock sur S3 garantit la conservation immuable.
- Versioning activé permet de restaurer un fichier ou une image écrasée.

13.4 Exploitabilité

La stratégie PRA ne se limite pas qu'à des mécanismes techniques : elle inclut des procédures exploitables par les équipes.

- **Runbooks qui** détaillent les étapes de restauration pour chaque scénario.
- **Tests réguliers** de PRA planifiés (au moins une fois par semestre en PR).
- **Automatisation** de certaines restaurations (scripts Terraform + Ansible) pour réduire les délais humains.

13.5 Livrables

- Sauvegardes automatiques configurées sur RDS et S3.
- Snapshots manuels documentés avant changements majeurs.
- Buckets S3 avec versioning, Object Lock et politiques de cycle de vie.
- Lifecycle policies ECR pour gestion des images.
- Documentation PRA (scénarios, procédures, contacts, priorités).
- Runbooks d'exécution des restaurations.
- Rapports de tests PRA réalisés régulièrement.

14 . Synthèse de la réalisation & Axes d'amélioration futurs

Synthèse de la réalisation

Le projet **RHZORION** a permis de concevoir et déployer une plateforme **SaaS RH cloud-native** reposant sur AWS, pensée dès l'origine pour être **scalable, sécurisée et conforme**.

Les principaux apports de cette réalisation sont :

- **Fondations solides** : organisation multi-comptes, VPC segmentés, journaux immuables, IAM fédéré avec MFA.
- **Sécurité by design** : chiffrement KMS systématique, gestion centralisée des secrets avec rotation, durcissement des flux réseau, WAF et ALB pour la protection applicative.
- **Chaîne CI/CD DevSecOps** : intégration OIDC sans clés statiques, scans automatisés (SAST, IaC, conteneurs), signatures cryptographiques des artefacts, gates en production.
- **Plateforme applicative robuste** : workloads Kubernetes (EKS) sécurisés (pods non-root, IRSA, NetworkPolicies), base de données RDS PostgreSQL hautement disponible, Registry ECR sécurisé et auditable
- **Observabilité et PRA** : stack complète (Prometheus, Grafana, Fluent Bit, OTel), alertes et SLOs formalisés, sauvegardes multi-niveaux, tests réguliers de PRA.

Ce socle apporte **confiance, traçabilité et résilience**, tout en respectant les standards **ANSSI DevSecOps v1.0, AWS Well-Architected Framework et RGPD**.

Axes d'amélioration futurs

Afin de renforcer et d'enrichir la plateforme, plusieurs pistes pourraient être explorées :

1. Scalabilité et performance

- Étendre l'autoscaling horizontal/vertical sur EKS et RDS pour absorber des charges massives.
- Intégrer du caching distribué (Redis/ElastiCache) pour réduire la latence des services critiques.

2. Sécurité renforcée

- Mettre en place des politiques d'Admission Controllers Kubernetes (OPA/Gatekeeper, Kyverno) pour interdire automatiquement les déploiements non conformes.
- Déployer un SIEM centralisé (ex. Splunk, ELK) pour corréler les événements de sécurité et améliorer la détection d'incidents.

3. CI/CD et supply chain

- Étendre la pipeline CI/CD avec des tests dynamiques (DAST) et fuzzing pour accroître la sécurité.
- Automatiser davantage la gestion des dépendances (renouvellement des librairies, correctifs proactifs).

4. Gestion des environnements

- Isoler davantage les comptes AWS par domaine métier (micro-segmentation organisationnelle).
- Introduire un environnement de préproduction miroir (staging) pour valider les déploiements à grande échelle.

5. Fiabilité & PRA avancé

- Mettre en place un PRA cross-région complet, avec réplication automatique et restauration orchestrée.
- Simuler des scénarios de chaos engineering pour tester la résilience en conditions réelles.

Enzo Loppin-Antimi - Studi Promotion KAO